Binary Hacking: Cheats for Game Boy Wars 2

Thomas Hetebrüg, Leah Ludwig, Julius Hamel
Beginner's Practical, B.Sc. Computer Science
Heidelberg University, Germany
sw322@stud.uni-heidelberg.de, leah.ludwig@stud.uni-heidelberg.de, julius.hamel@stud.uni-heidelberg.de

***Abstract*—Game Boy Wars 2 is a turn-based strategy game, which was published in 1998 for the Game Boy Color. While Game Boy Wars 2 was only released in Japan, we found an English community translation ROM online, which we used to learn the games mechanics and write cheats for it. [1] For the easy appliance of the cheats, we wrote a python framework using pyboy, which grants the player a graphical user interface (GUI) to select wanted cheats. This framework has been uploaded to github.[2]**

I. General Gameplay and Memory Mechanics

A. Introduction into the gameplay

In Game Boy Wars 2 (GBW2) a red and a white team fight against each other. The teams can either be controlled by two persons in a local pass-and-play mode or in a single player mode playing against the computer. Each team can recruit 11 different units in their 'Industrial' or their 'Command Base' structures. Each unit has different static, such as health points (HP), movement and attack ranges and recruitment prices. There also are 'Commercial' tiles scattered around the map which can be occupied by one of a team and grant additional money every round. The game ends, when one of the teams 'Command Base' gets occupied by the opposing team or when a team's last active unit gets killed.

B. General Memory Mechanics

1) Units in Memory: The RAM of GBW2 contains a block of addresses where the stats for units are saved. The memory address space `0xC980` to `0xCC0F` is reserved for the red team and `0xCC10` to `0xCE9F` is for white's units. We will reference these addresses as the units 'base memory' going forward. After loading a game these addresses all hold `7f`, until a unit is bought. After unit creation, 16 bytes of data, for example `0xC980` to `0xC98F` get populated with the units stats. A freshly generated Infantry unit on the red team might look like this:

```
00 03 08 02 00 00 32 00 63 63 49 00 28
00 00 00
```

The first byte is the unit type. This differs between the red team and the white team. While a red Infantry has the byte `00`, a white Infantry is type `01`. Generally through all of the unit, the white unit type seems to be the red unit type plus one. The second and third byte hold the units coordinates, the unit of the memory above is standing on the tile in column 3 and row 8. The fourth byte contains `02` for all units that are ready to be moved and `03` for all units that are exhausted. Exhausted units are units that either already have been used in this turn or have just been recruited. The 6th byte states the type of tile the unit is standing on.

The table below shows a non exhaustive list of different tile types and corresponding bytes for the map Devilman.

Tile type	Byte
Red Commercial	32
White Commercial	33
Grass	3D
Forest	3F
Mountain	40

The units HP are saved in the ninth byte and the units ammunition are saved in the tenth. Units lose ammunition when moving or in combat and it can be resupplied by specific supporting units like Transports.

2) Structures in Memory: The memory addresses right after the units base memory starting at `0xCEA0` are for the different structures on the map. Each structure is represented in four bytes of data. The first byte depicts the structure type:

Structure Type	Byte
Red Command Base	3A
White Command Base	3B
Red Commercial	32
White Commercial	33
Red Industrial	30
White Industrial	31

The second and third byte hold the tile coordinate, similar as in the units memory, with the second byte representing the column and the third byte representing the row of the tile. The HP of the structures are saved in the fourth byte. While the HP is being displayed as between 0 and 400 when playing the game, it is saved between 0 and 40_{10} (0 to 24_{16}) in the memory, with the displayed HP being the memory HP multiplied by factor ten.

3) Selecting a Unit: When selecting or hovering over a unit it's stats get loaded from the memory address as described above to the addresses `0xC4D3` to `0xC4E2`. The example the selected units HP are now also present at the memory address `0xC4DB`. These addresses are being used in instructions of events that happen during a turn and the base memory will only be updated after the events of the turn. Therefore we will call these addresses the units 'active memory'. When a selected unit targets a unit of the opposing team to attack, the stats of the opposing unit are copied to the addresses `0xC517` to `0xC526`, which also is used as such active memory.

[1] https://romsfun.com/roms/game-boy-color/game-boy-wars-2.html
[2] https://github.com/juhamel/GBW2_patch_framework

4) Turn flag: Since the functions for the game mechanics are used by both teams, it was very important to find the memory address that flags which teams turn it is. To do this we wrote a small script that takes different memory dump inputs and looks for memory addresses, which hold the same value for all inputs if it is the red teams turn and a different input for all the inputs on a white teams turn. This skript is also in the github repository under the /tools directory. After using memory dumps for different situations like in and out-of combat and different tile positions the tool identified multiple suitable candidates. The one we used in our implementation is the address `0xDF6E`, which is `3D` on a red teams turn and `53` on a white teams turn.

II. The Cheat Framework

A. General implementation

As we stated earlier, the actions of both teams use the same functions, but just load different data inputs. For example there is only one function for the combat, which takes the values from the active unit memory addresses, which are populated differently based on which teams turn it is. This made it very complicated to write static cheats that are only valid for one team, which is why we decided on an implementation based on hooks that trigger if the Program Counter (PC) reaches specific addresses and manipulate the memory accordingly. To do this we used pyboy [3], which gives us an emulator to run the game and an API to manipulate the memory in python.

1) Installation and Usage: After cloning the GitHub repository, you can install the needed dependencies by running the following command in the root directory of the framework:

```
pip install -e .
```

To run the emulator, execute the following command from the root directory:

```
python3 -m gbc_patcher [PATH_TO_ROM]
```

If no ROM path is provided, the ROM can also be selected through the GUI.

III. The Cheats

A. Invincibility

1) Technical Background: As indicated earlier, the fights use the stats of the units that are in the units active memory. While the HP of a unit are being saved as a number between 0 and 99_{10} in memory they are displayed as a group of up to 10 individual units with a single digit HP 0-10_{10}. If an infantry unit has 10 HP the unit contains of 10 individual persons. When starting a fight, these double digit HP numbers are converted into the corresponding single digit representation the following way: The to-be-transformed value is loaded into register A and function `0x6231` is called, which is in bank four. We will use the representation of 04:6231 with bank:address going forward. This function checks if A is zero, which should not happen and then calls another function 00:09B7, in which the main calculation is done. This function loops through the value in A and adds 1 to the C register for every time you can subtract 100_{10} (64_{16}) from Register A, without getting a negative result. Since there are no units with more than 99 HP, we believe that this might be used to check for errors. Afterwards it checks how many multiples of 10_{10} ($0A_{16}$) are in the value of register A and returns back into the function starting at 04:6231. The rest of this function checks if C is indeed zero, increases the B register another time, loads this value into A and returns to the original caller instruction, where A can now be used as the transformed value. This means the displayed HP are calculated the following way:

$$\text{displayHP} = \left\lfloor \frac{\text{memoryHP}}{10} \right\rfloor + 1 \tag{1}$$

This transformation is used four times when initiating a combat, for the four following values:

a) `0xC60C`*:* The red unit's current HP
b) `0xC611`*:* The red unit's new HP
c) `0xC614`*:* The white unit's current HP
d) `0xC619`*:* The white unit's new HP

The combat initialization starts with loading the HP of the attacking unit from the active memory `0xC4DB` to `0xC60C` and the HP of the attacked unit from `0xC51F` to `0xC614`. However, if it is whites turn the values of `0xC60C` and `0xC614` get switched, so that the addresses are always mapped to the same team.

The calculation for the new HP for the attacking team starts in 00:26D4 and for the attacked team in 00:26E7. For the attacking team, we subtract the value in `0xC610` from the value of `0xC617` and for the attacked unit it is calculated by `0xC60F`-`0xC618`.

There are multiple routines involved in calculating the actual damage, but three stand out specifically:

a) 00:0916: This routine is a shift-and-add multiplication, which multiplies registers B and E into the HL registers

b) 00:097D: This routine is a shift-and-subtract division, which divides the HL registers by the BC registers.

c) 00:0AEE: This routine is called multiple time to calculate an offset for the values in the HL registers, by adding A to L and adding a potential carry flag to H.

The detailed combat calculation starts in 00:262A. Since the total damage is calculated by a subtracting `0xC610` and `0xC618` from `0xC617` and `0xC60F` and they are calculated in a similar matter, with the only difference being an addition of `05` to `0xC60F` we will describe the functionality with the names 'base damage' and 'damage reduction' going forward.

The base damage is calulated by loading a value from the Read Only Memory, which is dependent on the unit type of the subjected team. For the attacking unit damage this is `0xC4D3` and for the the attacked it is `0xC517`. This value can then be manipulated further, depending on the eleventh and twelfth byte of the opposing units active memory, which hold specific additional combat stats. This value is then loaded into the B register, before loading the opposing units HP into register E and calling the procedures 00:0916 and 097D,

[3] https://github.com/baekalfen/pyboy

which multiplies this previouly loaded ROM value with the opposing units HP and divides it by the fixed value of 63_{16} (99_{10}). This result is then being saved into the base damage memory address for the teams, with the value being loaded into `0xC60F` getting an additional increase of `05`, to give the attacking team a little bit more damage.

$$\text{baseDamage} = \lfloor \frac{unitTypeValue * opposingUnitHP}{99} \rfloor + \begin{cases} 5 & \text{for attacked team} \\ 0 & \text{otherwise} \end{cases} \quad (2)$$

The damage reduction is calculated in a similar manner. It starts by loading the tile type of its own units location and calculates an offset based off that. This offset is then used to load a ROM value, which will be multiplied with the previously calculated base damage and then divided by 100_{10} as well. This result is then loaded into the respective address of `0xC610` or `0xC618`.

$$\text{damageReduction} = \lfloor \frac{tileTypeValue * baseDamage}{100} \rfloor \quad (3)$$

These values are then used to calculate the units new HP values and loaded into the respective memory addresses of `0xC611` and `0xC619`.

$$\text{newHP} = \text{activeMemoryHP} - (\text{baseDamage} - \text{damageReduction}) \quad (4)$$

2) PyBoy Implementation: To implement unit invincibility we need to place different hooks. One hook on the function that loads the Hp from the active memory to the units base memory, which happens at 00:3A81 and then another hook for the respective teams, at 04:6190 for red and at 04:6199 for white.

The instruction at 00:3A81 is part of a loop that copies a unit's active memory to their base memory by repeated

LD A, (HL+)
LD (DE), A
INC DE

instructions, where HL references the units active memory and DE the base memory. The instruction at 00:3A81 is the corresponding LD (DE), A instruction, where the units HP is copied from either `0xC4DB` or `0xC51F`. As an additional safety to not copy unwanted values, we check if DE is in a predefined set of addresses, which correspond to the base memory HP address, calculated from

`0xC988` + i * 0x10 for i in range(41)

and

`0xCC18` + i * 0x10 for i in range(41)

If this condition matches, we use the PyBoy API to load the value referenced by DE into A, resulting in the

LD (DE), A

instruction effectively being

LD (DE), (DE).

While this hook is sufficient to logically block units from taking any damage, the units still take damage in the combat animation.

To make the combat animation match with the logic we added the two additional hooks at 04:6190 and 04:6199. These instructions are part of the logic, where the units new HP values (`0xC611` and `0xC619`) are being transformed from the memory representation to the display representation. They follow the format of

LD A, C611/C619
CALL 6231
LD C611/C610, A

Our hooks are set on the respective

LD C611/C619, A

instruction and load the units current HP calculated earlier from `0xC60C` and `0xC614` respectively. Through this hook we make sure that the units calculated new HP is the same as the old HP, which is important for the combat animation.

During combat, the animation loops by decreasing the current HP value until it matches the new HP value, with animations of individuals dying every time the current HP value gets decreased.

B. Money Manipulation

1) Technical Background: Since the maximum money of a team is 99999_{10} ($01869F_{16}$) there are three bytes needed to hold money of a team. We identified the following bytes as responsible for holding a teams money:

	Low Byte	Mid Byte	High Byte
Red Team	FFE6	FFE7	FFE8
White Team	FFE9	FFEA	FFEB

The 3 byte number of the money simply gets split into the three bytes per team. The maxmium money for a team would therefore be represented as

Low Byte	Mid Byte	High Byte
9F	86	01

Alternatively the total amount can be calculated the following way:

$$\text{totalMoney} = \text{highByte} * 65536 + \text{midByte} * 256 + \text{lowByte} \quad (5)$$

2) PyBoy Implementation: We wanted to give the player the possibility to manually manipulate their or the opponents money. To do this we added two input fields to set the money to or to add/subtract individual arbitrary inputs or preset options to add or subtract 1000, 5000 and 10 000. These new money value is then calculated by our script and written to the responsible bytes for the affected team.

C. Money Locking

1) PyBoy Implementation: To lock the money to the most recent amount, we wrote a function that checks if the current money value of a team is lower than the money at the last check. If so, we load the previous value back into the bytes which hold the teams money and into our current money variable in the function. After this check the variable that hold the money to compare is updated. This function is called on

every tick in our framework and therefore blocks decrease in money, making it possible to buy units for free while still receiving money from the commercial structures.

D. Infinite Moves

1) Technical Background: As already stated in Section I.B.1) Units in Memory, units are only allowed to do one move per turn and are in an 'exhausted' state after their move. This state is represented by the fourth byte in the units base memory or in `0xC4D6` for a currently selected unit, which hold `02` for units that are ready to move and `03` for units that have already moved. This flag is set in 00:1C1C by the instruction

SET 0, (HL)

which sets the least significant bit in the byte referenced by the registers H and L, which is `0xC4D6` and therefore changes it from 10_2 (02_{16}) to 11_2 (03_{16}).

2) PyBoy Implementation: If a PyBoy hook is set it always triggers before the instruction is executed by the emulator. Therefore we had do set the hook for the infinite moves cheat on the instruction after 00:1C1C, which is 00:1C1E. If the hook would have been set on 00:1C1C the instruction would have overwritten any changes made by us and set bit 0 anyways. Therefore we set the hook at 00:1C1E, to trigger right after byte in `0xC4D6` has been changed and reset it back to `02`. Because this instruction is used by both units, we also added additional checks whose teams turn it is and if the cheat is activated for the team, to only reset the `0xC4D6` byte on the turn of the team with an active infinite moves cheat.

Since the hook only affects the currently selected unit in the active memory we also added another function. As soon as the cheat gets activated, the whole unit base memory block for the team gets checked for any `03` bytes in the fourth byte of each unit to reset them to `02`. Similarly to the implementation of the invincible units cheat, we also defined sets with the memory addresses for both teams the following way:

frozenset(0xC983 + i * 0x10 for i in range(41))

and

frozenset(0xCC13 + i * 0x10 for i in range(41))

E. Instant Capture

1) Technical Background: Capturing enemy structures is a crucial part of GBW2 and as we already introduced in Section I.B.2) Structures in Memory the stats for structures are saved in 4 byte blocks starting at the address `0xCEA0`. The HP of the structure are saved in the fourth byte of each block. When a structure is besieged, the current HP is loaded into the A register and the previously calculated damage is subtracted from it in 01:6C65. The damage to the structure is dependent on the besieging units HP and is calculated by the same procedure 00:09B7 used to transform a units memory HP to the display HP. The damage calculation can be described as the following equation:

$$\text{structueDamage} = \left\lfloor \frac{\text{attackingMemoryHP}}{10} \right\rfloor + 1 \qquad (6)$$

It is important to note here again, that the displayed HP of a structure is their HP in memory multiplied by factor 10. So the displayed structure Damage can be calculated by:

$$\text{displayedStructueDamage} = \lfloor \text{attackingMemoryHP} \rfloor + 10 \quad (7)$$

If the besieged structures HP hit 0 or below, the structure is conquered by the attacking team. During the conquer procedure the attacking units tile type gets loaded to register A, function 01:6CCA is called and the game checks the tile type and loads the corresponding conquered tile type from the ROM into the structures base memory and the units active memory. The new tile type is loaded from `0x6CDA` plus an offset, which is calculated from the previous tile type and `0xFFD6`, which seems to be another turn flag in the game, being `00` if red conquers and `01` if white conquers. After the structures team byte has changed the standard HP get loaded into the structures base memory. The HP are also loaded from the ROM, depending on an offset calculated by the structure type in the sub-routine 00:2148.

2) PyBoy Implementation: To implement a cheat that lets a unit instantly capture a structure we set a hook on instruction 01:6C65, which is responsible for subtracting the damage from the current health. Once this hook triggers, we check the turn flag to see which team is besieging the structure and if the cheat is enabled for this team. If this aligns, we use the PyBoy API to load the value from register A, which is the current structure HP into register C, which hold the calculated damage. Using this technique the incoming damage is always as high as the structure HP and the structure will be conquered through the normal game mechanic.